



IEE review

JANUARY 1992



The Taoist way
distributing system software (p.13)

Distributing system software

the Taoist way

Reliance on standardisation dominates operating-system development. But if the power of parallel processing is to be realised, novelty and innovation are still needed

by Roger Dettmer

Continuing progress in computing, in the long run, depends on making a success of parallel processing. In principle, the requirements for such success are simple: you need a basic hardware building block optimised for working with other similar blocks, and software that will use arrays of such blocks efficiently.

On the hardware front, progress has been steady, with the Irmos transputer playing a pioneering role. The software story is less satisfactory, and it is clear that the limitations of current software technology are a real obstacle to progress.

The reason is not hard to see. If parallel programming is to be worthwhile, then the overall computational load has to be spread across the available processors in a way that keeps them all reasonably busy. Programming a single processor is hard enough; coping with the added complexities of interactions between code running on multiple processors soon stretches the limits of human intelligence.

One possible approach to this problem is to follow the well established practice of the world of single-processor programming, and shield the parallel programmer from the complexity of the underlying hardware by an operating system. This could reduce the problem of parallel programming to manageable proportions by creating a simplified virtual machine, and organising load balancing across processors without intervention by the programmer.

There is a good deal of activity world-

wide in the development of such distributed operating systems. Academic examples include 'Amoeba' at Vrije Universiteit, Amsterdam, 'Meshis' at London's City University and 'Mach' at Carnegie-Mellon, Pennsylvania. Established commercial offerings include 'Chorus', from Chorus Systemes, and Perihelion's 'Helios'.

It is clear that the limitations of current software technology are a real obstacle to progress

All these examples share the same basic objectives - simplified code generation and efficient code execution on a network of processors. A distributed operating system that claims to take this objective to its ultimate conclusion is Taos, from Taos Systems, Ingatstone, Essex, which supports 'shrink-trapped' (i.e. binary-portable) parallel software running on a network of mixed, heterogeneous processors of arbitrary configuration.

Virtual processor

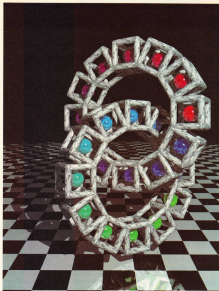
There are a number of interesting and novel ideas behind Taos; the best place to

start examining these is with the virtual processor (VP) - the secret of Taos's binary portability and support for mixed networks.

Applications to run under Taos are written either in C or assembly language. In the latter case, the target processor is a virtual machine, which has been specified as part of the system. The VP has all the attributes of a real processor - it's a 32-bit-word machine, has 36 registers, and supports standard data types and addressing modes. The only real difference between the VP and more conventional processors is that you can't get the VP in silicon.

Once written, Taos applications are either compiled (for C programs) or assembled (for assembly-language code) into the machine code of the VP. At run time, when the code is loaded onto a processor for execution, it is translated from VP machine code into that of the host processor. Currently, translators are available for the Irmos T800 and the Texas Instruments TMS320C40, so the same binary image of a program (in VP machine code) can be run, without any alteration, on a network of T800s, a network of C40s, or a heterogeneous network comprising both processors.

A second distinguishing feature of Taos is in its reliance on object-oriented programming (OOP). Interest in OOP has grown in recent years, primarily as a means of promoting software reusability. An object-oriented program comprises a set of reusable software 'objects' that communicate with each other by passing



This 740 x 578 pixel 24 bit colour image containing over 500 objects was generated on an array of eight transputers in under 11 minutes

messages. Objects and messages correspond to procedures/subroutines and data in conventional programming, with the important distinction that the inner workings of an object are totally hidden from the rest of the program. For reusability, objects must have no effect on program variables other than those specifically requested via a message.

Execution model

Taos programs comprise two basic types of objects: tool and control. Tool objects correspond loosely to conventional programming procedures, while control objects bind the tools into complete programs. A closer look at the execution model of a Taos program will clarify the roles of the two object types.

Program execution starts with the program's main control object being assigned to a convenient processor

within the network. The operating system uses the control object first to define the hardware resources required by the program (eg, stack requirements, mailboxes for passing messages between objects), and secondly to identify the main tool object of the program. The latter is then brought into the processor, and translated into the processor's machine code. Program execution proceeds as the main tool, in turn, calls subsidiary tools.

Execution continues in this way – as a single thread of sequential code within a single processor – until either the program terminates, or a tool calls a further control object, initiating a second sequential thread of computation. This second thread can either run on the original processor, or migrate to another processor for parallel execution. In general, Taos programs will comprise

parallel threads of sequential code running in parallel on multiple processors.

A natural affinity with parallelism isn't the only departure of the Taos execution model from conventional programming. Conventionally, programs are compiled and linked, or bound, into a single contiguous block before execution. This can cause a lot of work when making even small changes, as the whole program has to be recompiled and relinked. Taos takes a radically different approach: objects are held as unbound, independent entities on disc, and are not linked into the thread of execution until they are brought into the host processor. In effect, Taos applications are created 'on the fly', with code only being called on a strictly 'as-needed' basis, ensuring highly efficient use of the processor's computational and memory resources.

There are two other efficiency-boosting refinements. Objects can be designated as either non-virtual or virtual, the labels effectively defining first- and second-class status with respect to memory use. Non-virtual objects are brought into the host processor at the earliest opportunity, and remain there until the execution thread terminates; virtual objects are brought in at the last moment, and deleted almost immediately after use.

Lastly, Taos saves on memory by, whenever possible, sharing objects between execution threads. When a call to an object is made, the operating system checks whether a copy of the object is already in the processor's memory. If so, then that copy is used. It's a bit of a mouthful, but Taos objects are binary-portable, unbound and multi-threadable.

Load balancing

I have referred casually to execution threads being 'assigned' and 'migrating' to convenient processors, with the tacit assumption that such placements are consistent with efficient load balancing. In fact there can be nothing casual about the approach to load balancing: either it is efficient, or the whole rationale for parallel processing collapses.

Any distributed operating system worthy of the name will include some provision for automatic load balancing,

**In the world of
operating systems
the marketing power
of established
companies and the
movement towards
standardisation are
so all-pervading that
novelty, no matter
how ingenious, can
seem a positive
disadvantage**

The standard approach is to use a master program to keep track of the current load on the network, and to place new work in a way that keeps the network efficiently balanced. The problem is that, as the network becomes larger, the overhead of this master program becomes greater, consuming more of the system resources and constraining the ability of the network to expand.

Taos takes a different approach to load balancing: the network is self-balancing, with the balancing function distributed across all processors in the network. Running on every processor in a Taos network is a copy of the Taos micro-kernel. This provides all the standard functions common to any operating-system kernel (eg. memory and process management) together with a number of functions specific to Taos, including distributed load balancing. It works like this. When a new execution thread, or process, is started by an existing 'parent' process, the parent's kernel compares the load of the parent processor against that of other local processors, and hands the request to start the child process to the most appropriate processor. This allocation takes account of both the computational power of processors and the communications bandwidth of network links. The processor receiving the child, in turn, tries to balance its load locally, and so on until the child finds a suitably balanced location. In general, this will be close to its parent.

This self-balancing approach makes for overall load balancing plus messaging efficiency, regardless of network topology or network size. If required, the programmer can override automatic placement and place processes at specific locations.

Applications

To survive, and even thrive, computing innovations need to offer users the ability to do something that cannot be done, either conveniently or at acceptable cost, by established means. The success of the PC began with spreadsheets; the Macintosh owes its survival to desktop publishing.

Hopes for Taos are pinned on the computer-visualisation market, and the

first Taos application is a ray-tracing package, which is claimed to provide an order-of-magnitude improvement over the performance of existing systems. Ray tracing is an approach to the computer generation of photorealistic images that models, for each pixel, the ray traced from the pixel – possibly via multiple reflections and refractions – back to the originating light source. As a graphics algorithm it is extremely computation intense, but it is ideally suited to parallel programming as the calculations for each pixel can be performed independently.

The first Taos ray-tracing package is based on a standard PC and a plug-in card of T800 transputers. The package supports a wide range of standard graphics features – eg. importing three-dimensional objects from standard CAD packages, a comprehensive set of object primitives (planes, spheres, cones, ellipsoids etc.), plus any number of light sources (of any colour) – and can generate a photorealistic image in a matter of minutes. Current ray tracers typically take tens of hours for the same task, with the result that just a few seconds of computer-generated animation can involve weeks, if not months, of effort.

The performance of the Taos ray tracer should be significantly boosted by the availability of boards/modules for the TMS320C40. In theory, a ray tracer based on an array of 50 C40s should be able to generate video images in real time, which could revolutionise the existing simulation business as well as transforming the present crude imaging standards available in virtual-reality systems.

Taos is not, however, intended as a specialised operating system. It may be ideal for computer graphics, but it has

been designed for use across the full spectrum of networked systems, from embedded controllers through the multi-user, multi-tasking Unix box to the massively parallel supercomputer. The secret of success in all these areas is software availability; but this does not mean that the wider success of Taos will depend on the wholly unrealistic assumption that program developers will rush to write applications for a new operating system. Software is increasingly written to a well defined application/programmer interface (API), which defines a set of services provided by the underlying operating system and hardware. The basic requirement for running such software under Taos is to develop a set of Taos objects that implement these services on a network. Objects for APIs compliant with leading operating systems, especially variants of Unix, will be developed during 1992.

Prospects

Taos is certainly ingenious, and it is supported by a credible marketing plan, but in the world of operating systems the marketing power of established companies and the movement towards standardisation have become so all-pervading that novelty, no matter how ingenious, can seem a positive disadvantage. What chance of Taos thriving then? There's at least one precedent to suggest that its prospects are good.

The origins of Taos lie in the world of computer games. Chris Hinsley, the main architect of Taos, used to earn a living as a successful writer of games for machines like the Atari and the Amiga. It was the need to port such games between different systems, combined with the problems of managing large numbers of screen objects, that led to the VP and Taos's reliance on OOP. Not an especially promising background, you might say; but it has succeeded before. Back in the late 1960s, Ken Thompson, a computer scientist at Bell Labs, was trying to port a video game called Space Travel onto a DEC PDP-7. In the course of this work he developed a new file system and a rudimentary kernel – in fact, the germ of a new operating system. The name eventually chosen for this operating system was Unix.